
Application de Dessin Vectoriel

Projet Architecture Logicielle 2025-2026

Patrons de Conception et Principes S.O.L.I.D.

Projet d'Architecture Logicielle

Master 1 GIL — 2025/2026

Informations du projet

Responsables :	Kherbachi Rima, Mouhoubi Leïticia
Formation :	Master 1 GIL
Université :	Université de Rouen Normandie
Année :	2025 – 2026
Date :	Mai 2026

Table des matières

1	Présentation Générale	3
2	Architecture Globale	4
2.1	Paquetages	4
2.2	Diagramme UML Global	4
3	Patrons de Conception	5
3.1	Patron COMPOSITE	5
3.1.1	Motivation	5
3.1.2	Justification du choix	5
3.1.3	Acteurs Instanciés	5
3.1.4	Diagramme Composite	6
3.1.5	Code Composite	6
3.2	Patron VISITEUR	6
3.2.1	Motivation	6
3.2.2	Justification du choix	7
3.2.3	Acteurs Instanciés	7
3.2.4	Diagramme Visiteur	7
3.2.5	Code Visiteur	7
3.3	Patron FAÇADE	8
3.3.1	Motivation	8
3.3.2	Justification du choix	8
3.3.3	Acteurs Instanciés	8
3.3.4	Diagramme Façade	9
3.3.5	Code Façade	9
4	Patrons Écartés	11
4.1	Patron Observateur	11
4.2	Patron Monteur (Builder)	11
4.3	Patron Singleton	11
5	Schéma XML (DTD)	13
6	Respect des Principes S.O.L.I.D.	14
6.1	Single Responsibility (SRP)	14
6.2	Open/Closed (OCP)	14
6.3	Liskov Substitution (LSP)	14
6.4	Interface Segregation (ISP)	14
6.5	Dependency Inversion (DIP)	14

Conclusion**16**

Chapitre 1

Présentation Générale

Ce projet consiste à concevoir et implémenter trois programmes interdépendants pour manipuler des dessins vectoriels :

- **edit** : Éditeur graphique interactif permettant de créer et modifier des dessins
- **merge** : Outil de fusion de deux fichiers vectoriels
- **v2bmp** : Convertisseur de fichiers vectoriels en images PNG

L'objectif architectural est de concevoir une application modulaire, évolutive et maintenable en appliquant les principes S.O.L.I.D. et trois patrons de conception : **Composite**, **Visiteur** et **Façade**.

Le format de sauvegarde utilise XML avec extension **.vec**, autorisant l'imbrication de groupes de formes pour créer des structures complexes.

Chapitre 2

Architecture Globale

2.1 Paquetages

L'application est organisée en cinq paquetages métier :

- `fr.univrouen.draw.model` : Cœur métier - formes géométriques et structure d'arbre
- `fr.univrouen.draw.persistence` : Sauvegarde/chargement XML
- `fr.univrouen.draw.rendering` : Rendu graphique Java2D/Swing
- `fr.univrouen.draw.facade` : Interface simplifiée
- `fr.univrouen.draw.app` : Points d'entrée (main)

2.2 Diagramme UML Global

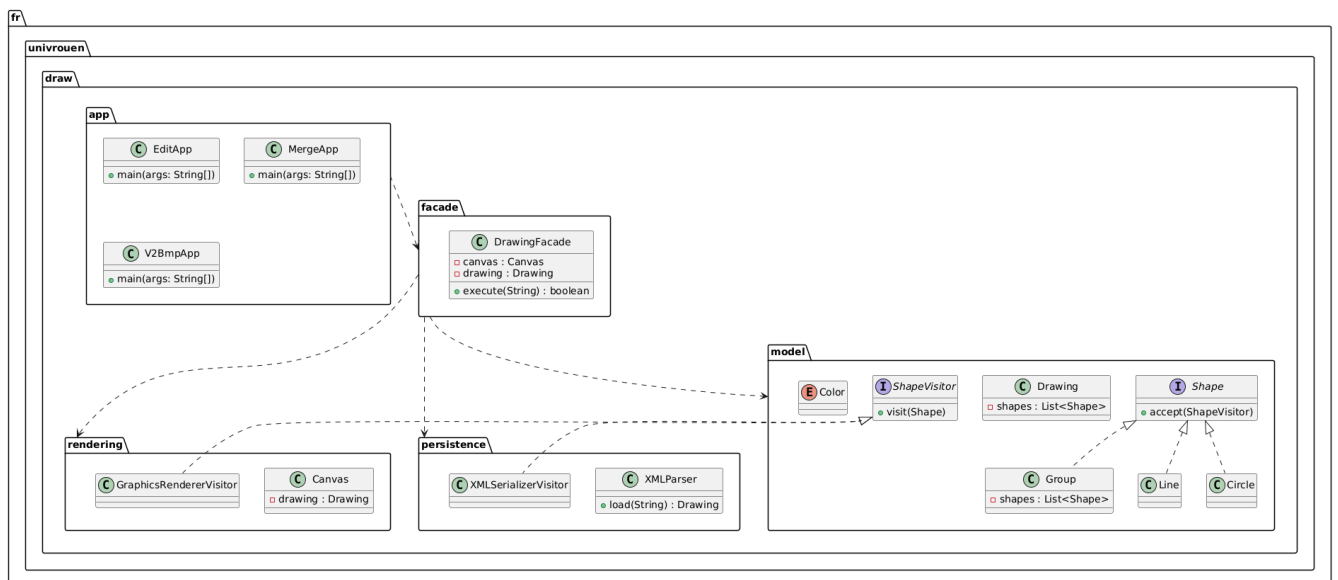


FIGURE 2.1 – Diagramme UML global

Chapitre 3

Patrons de Conception

3.1 Patron COMPOSITE

3.1.1 Motivation

Le dessin vectoriel manipule des primitives (Line, Circle, Rectangle, Ellipse) et des groupes pouvant contenir d'autres formes (y compris d'autres groupes). Le patron **Composite** permet de traiter uniformément les feuilles (primitives) et les branches (groupes) via une interface commune **Shape**.

3.1.2 Justification du choix

Le choix du patron Composite s'est imposé comme la seule solution viable car notre structure de données est par nature hiérarchique (un arbre de profondeur N). Sans ce patron, le code client aurait dû vérifier en permanence le type de l'objet manipulé (en utilisant des instructions `instanceof`) pour savoir s'il manipule une forme unique isolée ou une liste de formes. Le Composite élimine totalement cette complexité conditionnelle en offrant un contrat commun unique qui abstrait la notion de "composition", simplifiant drastiquement le parcours de l'arbre.

3.1.3 Acteurs Instanciés

- *Component* → **Shape** (interface)
- *Leaf* → Line, Circle, Rectangle, Ellipse
- *Composite* → Group

3.1.4 Diagramme Composite

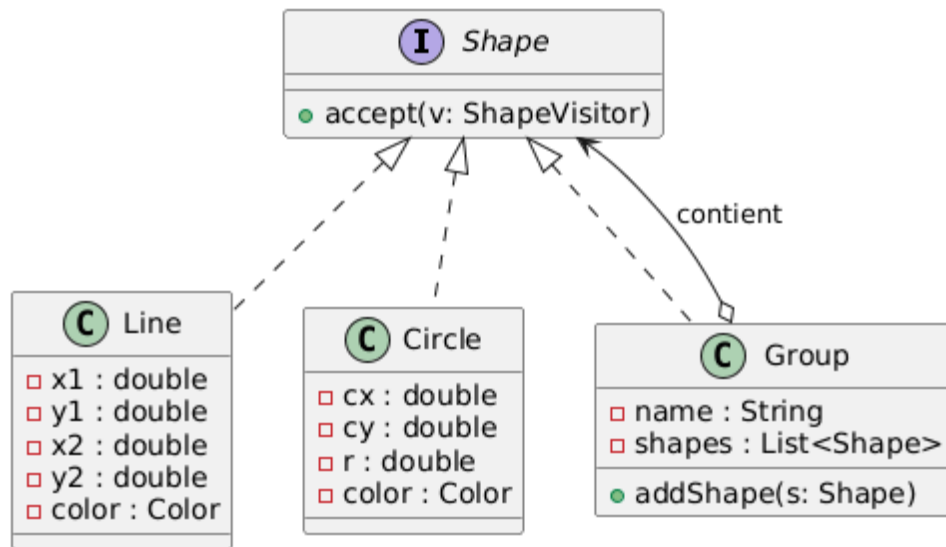


FIGURE 3.1 – Diagramme UML Composite

3.1.5 Code Composite

```

1 public class Group implements Shape {
2     private String name;
3     private List<Shape> shapes = new ArrayList<>();
4
5     public void addShape(Shape shape) {
6         shapes.add(shape);
7     }
8
9     @Override
10    public void accept(ShapeVisitor visitor) {
11        visitor.visit(this);
12    }
13 }
  
```

Listing 3.1 – Classe Group

3.2 Patron VISITEUR

3.2.1 Motivation

Sans le visiteur, ajouter des méthodes `draw()` ou `toXml()` à chaque forme polluerait le modèle et violerait l'OCP. Le **Visiteur** externalise ces opérations (rendu, sérialisation) dans des classes dédiées, permettant d'ajouter de nouveaux traitements sans modifier les formes.

3.2.2 Justification du choix

Dans notre système, la hiérarchie des formes géométriques est très stable (les concepts mathématiques d'une ligne ou d'un cercle ne changent pas). En revanche, les opérations appliquées sur ces formes (exporter en XML, exporter en JSON, dessiner en Java2D, dessiner en SVG) sont vouées à évoluer continuellement. Si nous avions utilisé le polymorphisme classique (ou le patron Stratégie), nous aurions dû modifier le code de toutes les classes du modèle à chaque nouveau format d'export. Le Visiteur exploite le mécanisme de "double dispatch" pour exécuter le bon algorithme en fonction du type de la forme, déplaçant ainsi la logique métier changeante dans des classes séparées, respectant parfaitement l'OCP (Open/Closed Principle) et le SRP (Single Responsibility Principle).

3.2.3 Acteurs Instanciés

- *Visitor* → ShapeVisitor
- *ConcreteVisitor* → XMLSerializerVisitor, GraphicsRendererVisitor
- *Element* → Shape

3.2.4 Diagramme Visiteur

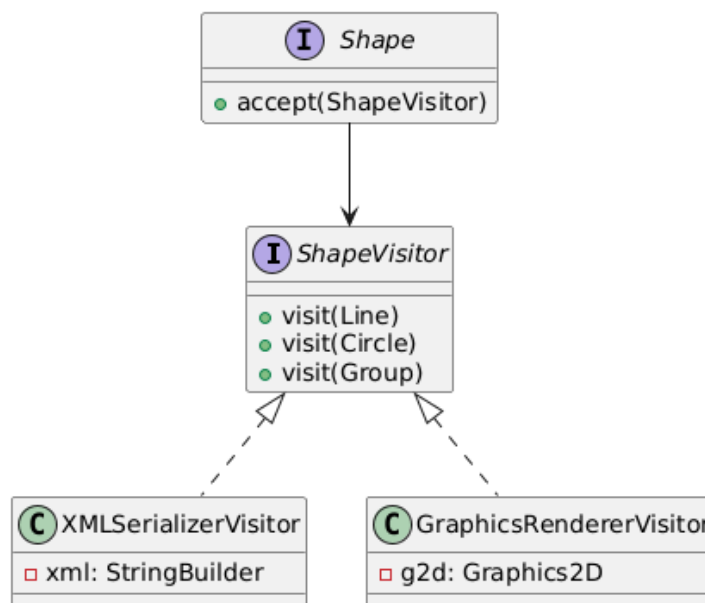


FIGURE 3.2 – Diagramme UML Visiteur

3.2.5 Code Visiteur

```

1 public class XMLSerializerVisitor implements ShapeVisitor {
2     private StringBuilder xml = new StringBuilder();
3
4     @Override
5     public void visit(Line line) {
6         xml.append(String.format(

```

```

7         "<line x1=\"%2f\" y1=\"%2f\" x2=\"%2f\" y2=\"%2f\"
           color=\"%s\" />\n",
8         line.getX1(), line.getY1(), line.getX2(),
9         line.getY2(), line.getColor().name());
10    }
11
12    @Override
13    public void visit(Group group) {
14        xml.append("<group name=\"").append(group.getName()).append
15            ("\">\n");
16        for (Shape shape : group.getShapes()) {
17            shape.accept(this);
18        }
19        xml.append("</group>\n");
20    }

```

Listing 3.2 – XMLSerializerVisitor

3.3 Patron FAÇADE

3.3.1 Motivation

Isoler l'interface utilisateur (CLI) de la complexité du sous-système. La **Façade** masque les détails de création des formes, mise à jour du rendu et persistance XML.

3.3.2 Justification du choix

L'application client (**EditApp**) doit interagir simultanément avec le modèle en mémoire (**Drawing**), le moteur de rendu AWT (**Canvas**), et le système de fichiers externe via le parseur XML. Laisser le client instancier, lier et coordonner tous ces éléments aurait généré un couplage excessivement fort. Le choix de la Façade s'est avéré indispensable pour établir une frontière stricte : le client se contente de transmettre des chaînes de caractères (les lignes de commande), et la Façade agit comme un chef d'orchestre sécurisé, encapsulant toute la logique interne complexe et protégeant le client des changements structurels futurs.

3.3.3 Acteurs Instanciés

- *Facade* → **DrawingFacade**
- *Subsystem* → **Drawing, Canvas, XMLParser**
- *Client* → **EditApp**

3.3.4 Diagramme Façade

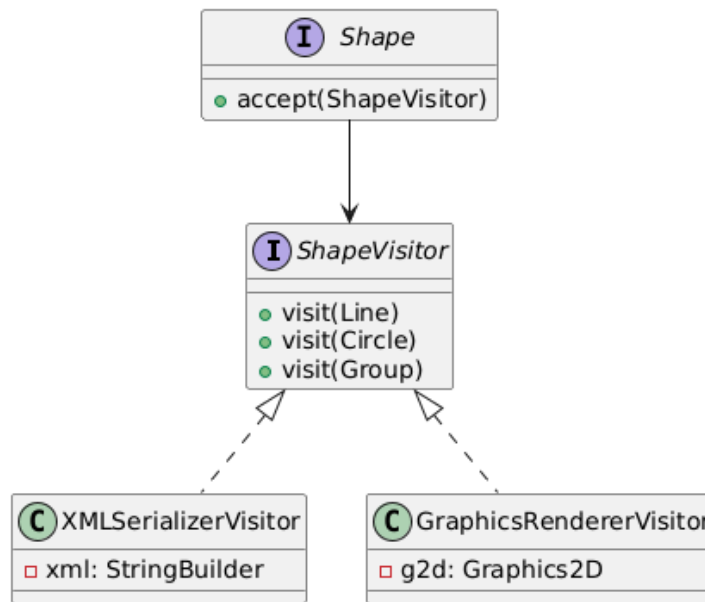


FIGURE 3.3 – Diagramme UML Façade

3.3.5 Code Façade

```

1 public class DrawingFacade {
2     private Canvas canvas;
3     private Drawing drawing;
4
5     public boolean execute(String commandLine) {
6         String[] parts = commandLine.trim().split("\\s+");
7         String cmd = parts[0].toLowerCase();
8
9         switch (cmd) {
10             case "new":
11                 drawing = new Drawing();
12                 canvas.setDrawing(drawing);
13                 break;
14             case "load":
15                 drawing = XMLParser.load(parts[1]);
16                 canvas.setDrawing(drawing);
17                 break;
18             case "save":
19                 XMLSerializerVisitor serializer = new
20                     XMLSerializerVisitor();
21                 for (Shape shape : drawing.getShapes()) {
22                     shape.accept(serializer);
23                 }
24                 break;
25             case "quit":
26                 return false;
27         }
28     }
29 }
  
```

```
26         }  
27         return true;  
28     }  
29 }
```

Listing 3.3 – DrawingFacade

Chapitre 4

Patrons Écartés

Lors de la conception, d'autres patrons de conception classiques étudiés en cours ont été envisagés puis écartés afin de privilégier la simplicité et d'éviter la sur-ingénierie (over-engineering) :

4.1 Patron Observateur

Contexte : Mettre à jour l'interface graphique (**Canvas**) à chaque fois que le modèle de données (**Drawing**) subit une modification (ajout ou suppression d'une forme).

Justification : Le patron Observateur aurait permis au **Drawing** (Sujet) de notifier automatiquement le **Canvas** (Observateur). Cependant, notre application fonctionne de manière séquentielle et synchrone via des commandes textuelles. La **Façade** joue déjà le rôle de chef d'orchestre : c'est elle qui, après avoir modifié le modèle, demande explicitement à la vue de se rafraîchir. Mettre en place un mécanisme d'abonnement/notification aurait inutilement complexifié le code et dilué le flux de contrôle sans apporter de réelle plus-value.

4.2 Patron Monteur (Builder)

Contexte : Faciliter la création et l'initialisation des formes géométriques qui possèdent plusieurs attributs (coordonnées x/y, rayon, couleur, etc.).

Justification : Le patron Monteur est excellent pour construire des objets complexes ou paramétrables étape par étape, en masquant la complexité de leurs constructeurs. Toutefois, dans notre projet, les primitives géométriques (**Line**, **Circle**, **Rectangle**) sont des structures de données relativement simples nécessitant au maximum cinq paramètres de base. L'implémentation d'une interface **ShapeBuilder** et de ses classes concrètes aurait généré beaucoup de code répétitif et lourd (boilerplate) par rapport à l'utilisation de simples constructeurs.

4.3 Patron Singleton

Contexte : Garantir qu'il n'existe qu'une seule et unique instance du **Drawing** (le dessin en cours) ou de la **DrawingFacade** durant toute l'exécution du programme.

Justification : Bien qu'il paraisse pratique pour accéder facilement aux éléments centraux, le Singleton est souvent considéré comme un anti-patron car il introduit un état global caché. Cela crée un couplage fort entre les classes et rend les tests unitaires très difficiles, ce qui viole directement le principe d'Inversion des Dépendances (DIP) de S.O.L.I.D. Nous avons préféré instancier les objets normalement dans le `main` et les passer par référence (injection de dépendance manuelle), ce qui maintient une architecture saine.

Chapitre 5

Schéma XML (DTD)

Le format de sauvegarde (.vec) repose sur une DTD autorisant l'imbrication des groupes :

Listing 5.1 – draw.dtd

```
<!ELEMENT drawing (line|rectangle|circle|ellipse|group)*>
<!ELEMENT group (line|rectangle|circle|ellipse|group)*>
<!ATTLIST group name CDATA #IMPLIED>

<!ELEMENT line EMPTY>
<!ATTLIST line x1 CDATA #REQUIRED y1 CDATA #REQUIRED
               x2 CDATA #REQUIRED y2 CDATA #REQUIRED
               color (red|green|blue|black|white) #REQUIRED>

<!ELEMENT rectangle EMPTY>
<!ATTLIST rectangle x1 CDATA #REQUIRED y1 CDATA #REQUIRED
                    x2 CDATA #REQUIRED y2 CDATA #REQUIRED
                    color (red|green|blue|black|white) #REQUIRED>

<!ELEMENT circle EMPTY>
<!ATTLIST circle cx CDATA #REQUIRED cy CDATA #REQUIRED
                 r CDATA #REQUIRED
                 color (red|green|blue|black|white) #REQUIRED>

<!ELEMENT ellipse EMPTY>
<!ATTLIST ellipse cx CDATA #REQUIRED cy CDATA #REQUIRED
                  rx CDATA #REQUIRED ry CDATA #REQUIRED
                  color (red|green|blue|black|white) #REQUIRED>
```

Chapitre 6

Respect des Principes S.O.L.I.D.

6.1 Single Responsibility (SRP)

Chaque classe a une seule raison de changer :

- **Shape** : Représenter une forme géométrique
- **GraphicsRendererVisitor** : Afficher avec Java2D
- **XMLSerializerVisitor** : Sérialiser en XML
- **XMLParser** : Parser XML
- **DrawingFacade** : Coordonner les commandes

6.2 Open/Closed (OCP)

Le code est **fermé à la modification** mais **ouvert à l'extension** :

- Ajouter une forme **Triangle** : créer une classe **Triangle**, aucune modification existante
- Ajouter un format JSON : créer un nouveau visiteur, aucune modification du modèle

6.3 Liskov Substitution (LSP)

Toute classe implémentant **Shape** est parfaitement substituable. Visiteurs et **Drawing** traitent uniformément **Line** et **Group**.

6.4 Interface Segregation (ISP)

Les interfaces sont *fin*es :

- **Shape** : une seule méthode `accept()`
- **ShapeVisitor** : méthodes de visite spécialisées

6.5 Dependency Inversion (DIP)

Les classes de haut niveau dépendent d'abstractions, non d'implémentations :

- **Drawing** utilise `List<Shape>`, pas `List<Line>`

— DrawingFacade dépend de Shape et ShapeVisitor

Conclusion

Ce projet démontre l'application concrète des patrons de conception et des principes S.O.L.I.D. en produisant une architecture :

- **Modulaire** : Paquetages avec responsabilités claires
- **Évolutive** : Ajout facile de formes et de visiteurs
- **Maintenable** : Code lisible et bien organisé
- **Testable** : Interfaces facilitant les tests unitaires

L'utilisation systématique des interfaces et des patrons Visiteur et Façade garantit un code propre, facilement maintenable et conforme aux bonnes pratiques d'architecture logicielle.